Exercício 11.1: Instalar o Argo CD e sincronizar a primeira Application

Aluno: Luiz Brito · **Data:** 24/10/2026 · **Disciplina:** Cloud Native Development (CP3)

Ambiente e adaptações

Enunciado	Feito	Motivo
minikube com Kubernetes 1.33	kind com 3 nós, Kubernetes **1.36.1**	mesmo cluster dos capítulos anteriores
Argo CD **v2.13.x**	Argo CD **v3.5.6**	a linha 2.13 está fora de suporte e não foi feita para o Kubernetes 1.36; a 3.5.6 é a estável atual
repositório público no **GitHub**	repositório público no **Gitea** dentro do cluster	escolha "só local" para o capítulo 11

O CLI `argocd` v3.5.6 foi baixado do release oficial com o SHA-256 conferido contra o `cli_checksums.txt`. As senhas (admin do Argo CD e do Gitea) foram geradas aleatoriamente e ficam em arquivos locais fora do git. Sequência completa em `02-commands.sh`.

Instalação e troca de senha

```
$ kubectl apply -n argocd --server-side --force-conflicts -f
https://raw.githubusercontent.com/argoproj/argo-cd/v3.5.6/manifests/install.yaml
$ kubectl -n argocd get pods
argocd-application-controller-0                      Running
argocd-applicationset-controller-58bb5c7479-x8xwz    Running
argocd-dex-server-dd6b85d9c-kk9f7                    Running
argocd-notifications-controller-6b85c65458-tbvw6     Running
argocd-redis-869b85489d-tpwtp                        Running
argocd-repo-server-5bf66565cd-nckw4                  Running
argocd-server-7474bb8f76-g6jx7                       Running

$ argocd admin initial-password -n argocd          # senha inicial (não reproduzida)
$ argocd login localhost:8081 --username admin --password <inicial> --insecure --grpc-web
$ argocd account update-password --current-password <inicial> --new-password <nova> --grpc-web
$ argocd login localhost:8081 --username admin --password <nova> --insecure --grpc-web
Context 'localhost:8081' updated
$ kubectl -n argocd delete secret argocd-initial-admin-secret
secret "argocd-initial-admin-secret" deleted from argocd namespace
$ kubectl -n argocd get secret argocd-initial-admin-secret
Error from server (NotFound): secrets "argocd-initial-admin-secret" not found
```

O secret inicial é apagado porque a senha nele continua válida para quem tiver leitura de Secrets em `argocd`, até ser trocada e removida.

Repositório `fiap-store-config`

Público no Gitea: `http://localhost:3000/fiap/fiap-store-config`, lido pelo Argo CD em `http://gitea-http.gitea.svc.cluster.local:3000/fiap/fiap-store-config.git`.

`base/deployment.yaml` leva o `securityContext` completo do capítulo 10:

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: store-api
spec:
  replicas: 1
  selector:
    matchLabels:
      app: store-api
  strategy:
    rollingUpdate:
```

```
      maxUnavailable: 0
      maxSurge: 1
  template:
    metadata:
      labels:
        app: store-api
    spec:
      automountServiceAccountToken: false
      securityContext:
        runAsNonRoot: true
        runAsUser: 65532
        runAsGroup: 65532
        fsGroup: 65532
        seccompProfile:
          type: RuntimeDefault
      containers:
        - name: store-api
          image: store-api:1.0
          imagePullPolicy: IfNotPresent
          ports:
            - name: http
              containerPort: 8000
          env:
            - name: APP_ENV
              value: dev
          readinessProbe:
            httpGet:
              path: /ready
              port: http
          livenessProbe:
            httpGet:
              path: /health
              port: http
          resources:
            requests:
              cpu: 50m
              memory: 64Mi
            limits:
              memory: 128Mi
          securityContext:
            allowPrivilegeEscalation: false
            readOnlyRootFilesystem: true
            capabilities:
              drop: ["ALL"]
          volumeMounts:
            - name: tmp
              mountPath: /tmp
      volumes:
```

```
        - name: tmp
          emptyDir: {}
```

`overlays/dev/kustomization.yaml`, como foi para o primeiro commit:

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
namespace: store-dev
resources:
  - ../../base
replicas:
  - name: store-api
    count: 2
images:
  - name: store-api
    newTag: "1.0"
```

Validado **antes** do commit:

```
$ kubectl kustomize overlays/dev | grep -E "namespace:|replicas:|image:"
  namespace: store-dev
  namespace: store-dev
  replicas: 2
        image: store-api:1.0
```

Application

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: fiap-store-dev
  namespace: argocd
  finalizers:
    - resources-finalizer.argocd.argoproj.io # apagar a Application apaga o que ela criou
spec:
  project: default
  source:
    # Gitea dentro do cluster (substitui o GitHub publico do enunciado)
    repoURL: http://gitea-http.gitea.svc.cluster.local:3000/fiap/fiap-store-config.git
    targetRevision: main
    path: overlays/dev
  destination:
    server: https://kubernetes.default.svc
    namespace: store-dev
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
      - CreateNamespace=true
```

O Kubernetes 1.36 avisa ao criar: `metadata.finalizers: "resources-finalizer.argocd.argoproj.io": prefer a domain-qualified finalizer name including a path (/)`. O nome é o que o Argo CD documenta e funciona, mas o aviso indica que finalizers sem `/` estão sendo desencorajados.

```
$ argocd app get fiap-store-dev --grpc-web
Name:               argocd/fiap-store-dev
Project:            default
Server:             https://kubernetes.default.svc
Namespace:          store-dev
Source:
- Repo:             http://gitea-http.gitea.svc.cluster.local:3000/fiap/fiap-store-config.git
  Target:           main
  Path:             overlays/dev
Sync Policy:        Automated (Prune)
Sync Status:        Synced to main (61c72a0)
Health Status:      Healthy

GROUP  KIND        NAMESPACE  NAME       STATUS   HEALTH   HOOK  MESSAGE
       Namespace               store-dev  Running  Synced         namespace/store-dev created
       Service     store-dev  store-api  Synced   Healthy        service/store-api created
apps   Deployment  store-dev  store-api  Synced   Healthy        deployment.apps/store-api created
```

Commit de 2 para 4 réplicas, sem tocar no cluster

Antes:

```
$ kubectl get pods -n store-dev -o wide
NAME                         READY   STATUS    RESTARTS   AGE   IP               NODE
store-api-6944799854-bl22q   1/1     Running   0          28s   192.168.163.71   fiap-store-control-
plane
store-api-6944799854-pr7nd   1/1     Running   0          28s   192.168.29.208   fiap-store-worker
```

Commit (único arquivo alterado):

```
--- a/overlays/dev/kustomization.yaml
+++ b/overlays/dev/kustomization.yaml
@@ -5,7 +5,7 @@ resources:
   - ../../base
 replicas:
   - name: store-api
-    count: 2
+    count: 4
```

```
commit a0948b0  2026-10-24T02:02:20-03:00 (05:02:20Z)  chore(dev): scale store-api to 4 replicas
```

Depois do push, o único comando no cluster foi um laço **só de leitura** (`kubectl get deploy ... readyReplicas`) esperando o 4. Nenhum `apply`, `scale` ou `argocd app sync`.

Depois:

```
$ kubectl get pods -n store-dev -o wide
NAME                         READY   STATUS    RESTARTS   AGE     IP               NODE
store-api-6944799854-7flxx   1/1     Running   0          33s     192.168.29.207   fiap-store-worker
store-api-6944799854-bl22q   1/1     Running   0          7m15s   192.168.163.71   fiap-store-
control-plane
store-api-6944799854-fcmnj   1/1     Running   0          33s     192.168.13.55    fiap-store-worker2
store-api-6944799854-pr7nd   1/1     Running   0          7m15s   192.168.29.208   fiap-store-worker

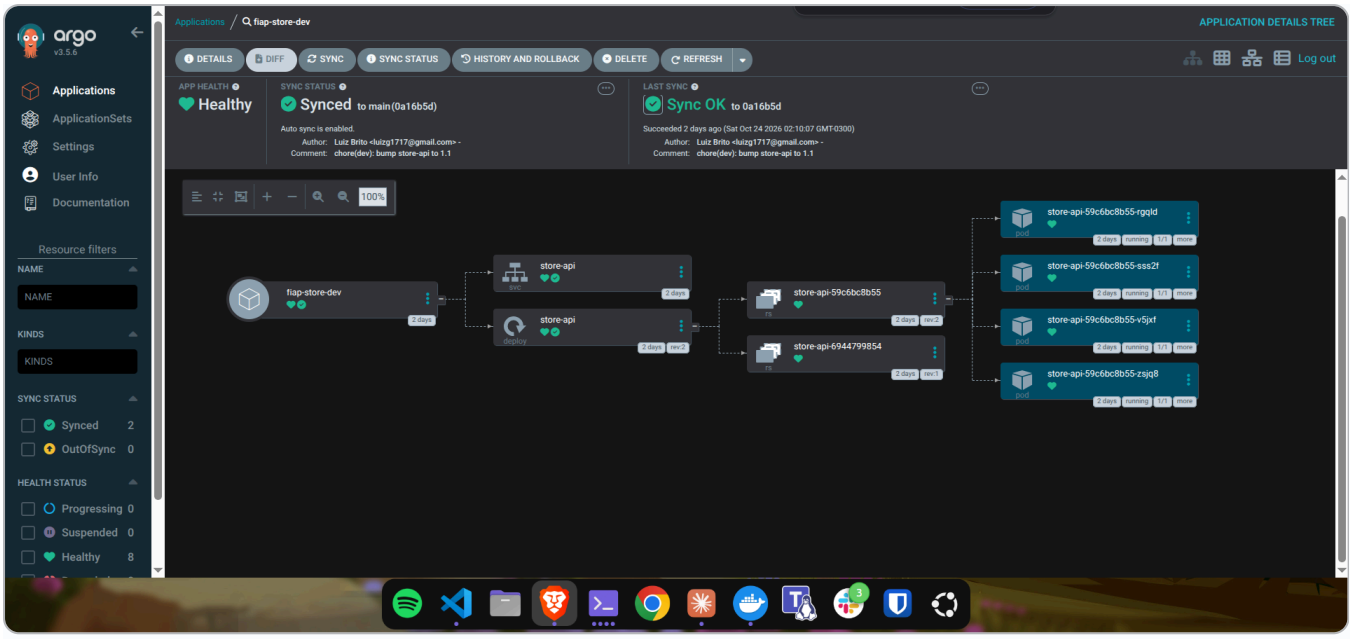
$ kubectl get pods -n store-dev --sort-by=.metadata.creationTimestamp -o custom-
columns=NAME:.metadata.name,CRIADO:.metadata.creationTimestamp
NAME                         CRIADO
store-api-6944799854-bl22q   2026-10-24T05:01:52Z
store-api-6944799854-pr7nd   2026-10-24T05:01:52Z
store-api-6944799854-7flxx   2026-10-24T05:08:34Z
store-api-6944799854-fcmnj   2026-10-24T05:08:34Z
```

Evento	Horário (UTC)
commit/push `a0948b0`	05:02:20
Pods novos criados	05:08:34
4/4 prontos	05:08:50

6 min 14 s entre o commit e os Pods novos. Os dois Pods novos usam o mesmo ReplicaSet (`6944799854`): só a contagem mudou, sem rollout. O Argo CD descobre o commit por **polling**, e a `argocd-cm` documenta `timeout.reconciliation: 120s` e `timeout.reconciliation.jitter: 60s` (entre 2 e 3 min por ciclo). O tempo medido passou desse máximo, provavelmente porque o primeiro ciclo depois do push ainda viu a revisão em cache. Não fui atrás da causa exata. Com um webhook do Git para o `argocd-server`, a reconciliação seria em segundos.

```
$ argocd app get fiap-store-dev --grpc-web
Sync Status:        Synced to main (a0948b0)
Health Status:      Healthy

$ argocd app history fiap-store-dev --grpc-web
ID      DATE                           REVISION
0       2026-10-24 02:01:52 -0300 -03  main (61c72a0)
1       2026-10-24 02:08:34 -0300 -03  main (a0948b0)
```



Critérios de aceite

Critério	Resultado
`Sync Status: Synced` e `Health Status: Healthy`	ok
4 Pods `Running` sem `kubectl apply` / `scale` depois da criação da Application	ok
`argocd app history` com pelo menos 2 revisões distintas	ok (`61c72a0`, `a0948b0`)